WHAT IS PLANKTOS?

Agent-based modeling in prescribed fluid flows

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part VI • Chapter 34

The central idea

Move many autonomous organisms through a resolved velocity field while retaining behavior, stochasticity, and collision geometry.

Source basis: Strickland et al. (2022), Hamlet et al. (2023), and Planktos commit `a1c2128` (2026-08-19); the plots are this series' own runs (`examples/Planktos`); the other pictures are two figures of Hamlet et al. (2023, CC BY 4.0) and one of Planktos's documentation.

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

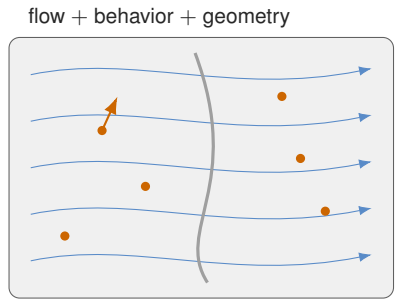
LEARNING GOALS

By the end, you should be able to:

1. state when a **one-way-coupled point-agent** model is defensible;
2. map a scientific question to an `Environment` and one or more `Swarm` objects;
3. choose among ODE, Itô SDE, and custom algorithmic behavior;
4. explain how Planktos interpolates flow data and enforces outer and immersed boundaries;
5. distinguish trajectory visualization, KDE, and FTLE diagnostics;
6. interpret the gorgonian capture example without overextending its assumptions;
7. say what changed between the 2022 paper and Planktos 1.0.3, and how this series' flows reach Planktos.

THE SCIENTIFIC PROBLEM: BEHAVIOR INSIDE TRANSPORT

- Marine and aerial plankton are carried by spatially and temporally varying flows.
- They may also swim, fly, align, settle, search, or respond to environmental cues.
- Obstacles and living structures reshape the flow and create collision or capture surfaces.
- The observable is often a **distribution**: dispersal, residence, encounter, or capture probability.



Scientific framing: Strickland et al. (2022), Sections 1 and 5. Schematic: instructor's course notes.

A COMPETITION OF VELOCITIES AND SCALES

A useful first diagnostic is the velocity ratio

$$V_r = \frac{U_a}{U_f},$$

where U_a is a characteristic active speed and U_f is a characteristic ambient-flow speed.

- $V_r \ll 1$: transport is mostly flow-driven.
- $V_r \sim 1$: behavior and advection compete strongly.
- $V_r \gg 1$: active motion may dominate locally.

Do not stop at one ratio

Also compare body size, obstacle spacing, correlation time, diffusivity, and observation time. V_r organizes regimes; it does not prove that a point-agent or one-way coupling is valid.

Velocity-ratio framing: Strickland et al. (2022), Section 1.

THE MODELING CONTRACT: ONE-WAY COUPLING

Assumption

The flow and immersed geometry influence the agents, but the agents do not alter either one.

Plausible when

- agents are small relative to resolved flow scales;
- concentrations are low enough to neglect collective forcing;
- wakes and body-induced disturbances decay before affecting other agents;
- agent shape can be reduced to a point trajectory for the question asked.

Questionable when

- dense suspensions actively mix the fluid;
- locomotion produces persistent wakes or stresses;
- deformable-body hydrodynamics controls motion;
- capture depends on unresolved finite-size contact physics.

Scope and limitations: Strickland et al. (2022), Abstract and Section 5.

THE POINT-AGENT EQUATION

One common model represented by Planktos is

$$d\boldsymbol{X}_t = \big[\boldsymbol{u}(\boldsymbol{X}_t, t) + \boldsymbol{b}(\boldsymbol{X}_t, t, \mathcal{S}_t)\big]\,dt + \boldsymbol{\sigma}(\boldsymbol{X}_t, t, \mathcal{S}_t)\,d\boldsymbol{W}_t.$$

- $\boldsymbol{u}$ prescribed fluid velocity, interpolated to the agent location;
- $\boldsymbol{b}$ active drift or deterministic behavior;
- $\boldsymbol{\sigma}\,d\boldsymbol{W}_t$ stochastic motion, interpreted as an Itô SDE by the built-in Euler–Maruyama route (the built-in solver restricts $\boldsymbol{\sigma}$; see the SDE slide);
- $\mathcal{S}_t$ agent state, neighbor information, or other model variables supplied by the user.

Important

This equation is a template, not a restriction: `apply_agent_model` may return positions from arbitrary code.

NONDIMENSIONAL CHECKS BEFORE SIMULATION

$$V_r = U_a/U_f,$$
$$\mathrm{Pe} = U_f L/D,$$
$$\mathrm{St} \sim \tau_p/\tau_f.$$

- Pe compares advective transport with diffusive spreading; D is the agents' effective diffusivity (Planktos's covariance is $\mathtt{cov} = 2D$).
- A particle response-time ratio helps decide whether tracer dynamics are adequate: τ_p is the particle's viscous response time, $\tau_f = L/U_f$ the flow time.

Numerical resolution checks

- Is Δt small compared with flow and behavior time scales?
- Can an agent cross a thin obstacle in one step?
- Is the CFD mesh fine enough for the gradients the behavior uses?

Stokes number of a small sphere

For radius a and $\tau_p = 2a^2/(9\nu)$: $\mathrm{St} = \frac{2}{9}(a/L)^2\mathrm{Re}$, $\mathrm{Re} = U_f L/\nu$, the form used by Planktos's inertial-particle model, where the density enters separately (slide 27).

St form: Haller & Sapsis (2008), as implemented in `motion.inertial_particles` (commit `a1c2128`).

WHAT PLANKTOS IS—AND IS NOT

Planktos provides

- vectorized 2D/3D agent simulation;
- prescribed steady or time-dependent velocity fields;
- ODE, Itô SDE, and custom behavior routes;
- immersed-mesh collision handling;
- plotting, animation, KDE, vorticity, and FTLE tools.

Planktos does not

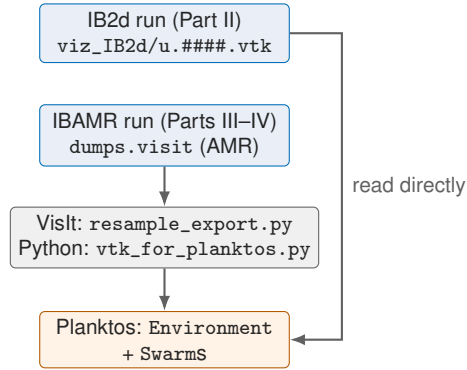
- solve the Navier–Stokes equations;
- feed agent forces back into the flow;
- automatically validate biological assumptions;
- replace convergence, uncertainty, or sensitivity studies.

Current source audited for this tutorial

Version 1.0.3, GPLv3, commit `a1c2128` (2026-08-19).

PLANKTOS IN THIS SERIES (1/2): FROM IB2D AND IBAMR RUNS

- The flows come from Parts II–IV. An IB2d run's `viz_IB2d/u.####.vtk` files are read as they are, from dump 1 on (pyIB2d writes dump 0's velocity transposed; chapter 35); an IBAMR run is resampled on a uniform grid by VisIt first (chapter 33), then its vector is made the active one by `vtk_for_planktos.py`, a plain Python script (chapter 35).



Chapters 33 and 35; `examples/analysis/resample_export.py` and `vtk_for_planktos.py`.

PLANKTOS IN THIS SERIES (2/2): EXAMPLES, PICTURES, CHAPTER 35

- `examples/Planktos` runs Planktos 1.0.3 on two of this series' flows: chapter 4f's starfish in a pulsed channel (2D, IB2d) and IBFE/10's flow tank with a plate (3D, IBAMR; chapter 22).
- The plots of this chapter are those scripts' output; the other pictures are two figures of Hamlet et al. (2023, CC BY 4.0) and one of Planktos's documentation. The 2022 paper's figures are described in words, with what they found (the article is not openly licensed).
- Chapter 35 is the how-to: installing, the calls these examples use, and the traps met on the way.

Chapters 4f, 22 and 35; `examples/Planktos/README.md` (the scripts, their times and results).

PLANKTOS SINCE 2017 (1/2): TO THE PAPER

when	what
2017–2018	W. C. Strickland starts Planktos (the GitHub repository dates from January 2017). Its documentation is cited as “Strickland, W.C. (2017)”; the lab’s 2021 deck cites the software as “Strickland, C. (2018), Planktos agent-based modeling framework”.
2020	Release 0.1 (February), the code of Ozalp et al. (2020, *Biomimetics* 5:2, brine shrimp); 0.2 (December), immersed boundaries in 2D and 3D.
2021	Releases 0.3 (June: the `planktos` package, FTLE, KDE in the plots, the Euler–Maruyama and RK45 solvers) and 0.4 (July: Sphinx documentation, sticky collisions). The lab’s “Getting started with Planktos” deck: Python 3.5+, FFmpeg and VTK from conda-forge, IBAMR data converted with VisIt’s Python 2.7; examples `example_2d.py`, `example_IBAMR_ibmesh.py`, … (their 2026 names: chapter 35).
2022	The paper: Strickland, Battista, Hamlet and Miller, *Bull. Math. Biol.* 84:72, the framework and its examples. The custom-behaviour method was then called `get_positions`; it is `apply_agent_model` now (renamed in 1.0.0).

What changed after the paper: next slide and slide 13.

Planktos `README.md`, `changelog.txt` and release notes v0.1–v0.4 (GitHub, read 17 October 2026); “Getting started with Planktos” (Miller lab, 2021), slides 7–20; Strickland et al. (2022).

PLANKTOS SINCE 2017 (2/2): AFTER THE PAPER

when	what
2022–2025	Releases 0.5 (April and October 2022, either side of the paper's publication: periodic agent boundaries, NetCDF, `pip install`, the flow kept as its splines in time, corrected inertial-particle equations), 0.6.2 (2023: a faster collision search, and a fix for long slides that tunnelled through mesh elements) and 0.7.0 (January 2025: `after_move`, property history, per-agent colours and headings).
2023	The gorgonian capture study (Hamlet et al., *J. Exp. Biol.* 226), slides 38 and 41–42.
2026	Releases 1.0.0 (30 June: moving 2D immersed boundaries; a refactor that breaks older scripts: `environment` and `swarm` became `Environment` and `Swarm`, `get_positions` became `apply_agent_model` without its `params` argument, the Boolean `ib_collision` became the index `ib_collision_idx`; a test suite), 1.0.1 (documentation; tests and a spell check on GitHub Actions, Python 3.8 and 3.12), 1.0.2 (immersed-boundary collision fixes) and 1.0.3 (`a1c2128`, 19 August: bug fixes that change two kinds of numbers, slide 13). National Science Foundation award DMS-2410988, 2024–2027.

Moving 2D meshes came after the paper, in 1.0.0 (slide 30); `calculate_FTLE` with a user's `Swarm` (one route to an agent-based FTLE like the paper's; the paper does not say which it used) was mended in 1.0.2. Results computed with an older version can differ from 1.0.3's for the reasons on slide 13.

Planktos `README.md`, `changelog.txt` and release notes v0.5–v1.0.3 (GitHub, read 17 October 2026); Strickland et al. (2022); Hamlet et al. (2023).

WHAT 1.0.2 AND 1.0.3 CHANGED, AND WHY OLD NUMBERS CAN DIFFER (1/2)

- FTLE (1.0.3): normalized by the time actually integrated, not by the requested one. They differ where a point of the finite-difference stencil leaves the domain early, through a `zero` boundary (the notes add "an immersed boundary", but in the code an immersed boundary never removes an agent); there the old value was too low, "0.34–0.72 of truth on an analytic field whose FTLE is exact and uniform". Interior values are unchanged only when $t_0 = 0$: 1.0.2 divided by `T` itself, the end time, so with $t_0 \neq 0$ every value changes by $T/(T - t_0)$ (a uniform strain of rate 1 with $t_0 = 0.5$, $T = 1$: 0.5 by 1.0.2's formula, 1.000 in 1.0.3).
- Velocity statistics in the plots (1.0.3): read from the recorded velocities instead of differences of positions; they differ for agents that hit a boundary. At the initial time the plots now show the agents' starting fluid drift, not zero.

Planktos release notes and `changelog.txt`, v1.0.3 (`a1c2128`), GitHub; the quotations are v1.0.3's; `Environment.calculate_FTLE` of v1.0.2 (divides by `T`) and v1.0.3 (by the elapsed time).

WHAT 1.0.2 AND 1.0.3 CHANGED (2/2)

- A failed step (1.0.3): a step that fails or is interrupted part way leaves the boundary conditions applied to some agents and not others (some may sit inside an immersed boundary); `envir.time` is now set to `None` to mark it, and the next `move()` refuses with instructions for backing the step out.
- Also in 1.0.3: a `Swarm` subclass that replaces `move()` instead of extending it now warns (behaviour belongs in `apply_agent_model`); `Environment.reset` clears the velocity and property histories too.
- 1.0.2: sliding collisions where three or more mesh elements meet; the collision back-off, which grew with the domain and gave NaN positions at negative coordinates; duplicated vertices (NaN positions, or a step that never finished); head-on hits (a spurious NumPy warning); `calculate_FTLE` with a user-supplied `Swarm` fixed (it raised `AttributeError` on the first step); meshes imported with `'adjacent'` or as moving meshes follow the fluid's coordinate shift.

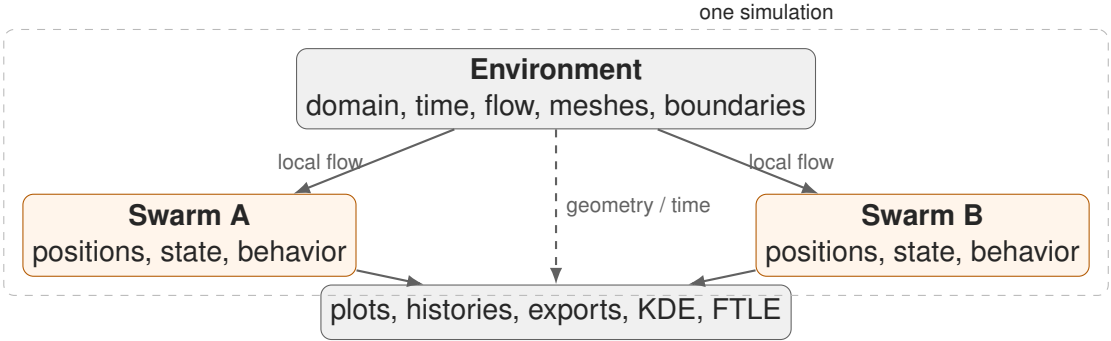
Planktos release notes and `changelog.txt`, v1.0.2 (`ccf4aa7`) and v1.0.3 (`a1c2128`), GitHub (the quotations of v1.0.3's notes are on the previous slide).

CITING PLANKTOS, THEN AND NOW

- 2021 (the lab's deck, after the repository's README of the time): "If you use this software in your project, please cite it as: Strickland, C. (2018), Planktos agent-based modeling framework."
- 2026 (the README): "If you use this software in your project, please cite the following paper": Strickland, W.C., Battista, N.A., Hamlet, C.L., Miller, L.A. (2022), *Bulletin of Mathematical Biology* 84(72). "Additionally, the documentation can be cited as" Strickland, W.C. (2017), https://planktos.readthedocs.io. A BibTeX entry for both is in `Planktos.bib` (which dates the documentation 2021; the README says 2017).
- Record with every result the version (`planktos.__version__`) and the commit; chapter 35's last capstone slide lists what else to keep (seeds, parameters, the flow's provenance).

Planktos `README.md` (October 2026); "Getting started with Planktos" (Miller lab, 2021), slide 7.

ARCHITECTURE: A WORLD CONTAINING POPULATIONS



- Multiple Swarms may share an Environment and carry different behaviors.
- Use `Environment.move_swarms` to advance all populations before the environment clock increments.
- Note that `move_swarms` defaults to `ib_collisions='sliding'` for every Swarm; pass `ib_collisions='default'` to honor each Swarm's own `ib_condition`.

Schematic: instructor's course notes; API from `planktos/_environment.py`, commit `a1c2128`.

THE ENVIRONMENT: EULERIAN INFORMATION

It owns the shared physical setting:

- rectangular 2D or 3D domain and agent boundary conditions;
- velocity components on a rectilinear grid and their time stamps;
- fluid metadata such as ρ, μ, ν, U, and a characteristic length;
- static or supported time-varying immersed meshes;
- simulation clock and registered Swarms.

It also performs

flow import, tiling and regridding, interpolation, vorticity and FTLE calculations, environment plotting, and coordinated Swarm movement (joint plotting of several swarms is not yet implemented).

Current API: `planktos/_environment.py`, commit `a1c2128`.

THE SWARM: VECTORIZED LAGRANGIAN STATE

- Positions and histories are stored in NumPy-style arrays, not one Python object per organism.
- Shared parameters live in `shared_props`; individual variation lives in the pandas table `props`.
- Each Swarm has one movement model and belongs to one Environment.
- A random seed can make stochastic runs reproducible.

Why group agents?

Vectorization improves computational efficiency and makes population-level plotting and statistics natural.

Cross-Swarm interactions

They are not a built-in sensing mechanism; custom subclasses can inspect `self.envir.swarms` when such coupling is required.

A MINIMAL CURRENT WORKFLOW

```
import planktos

env = planktos.Environment()       # becomes 3D on loading 3D data
env.read_comsol_vtu_data("velocity.vtu", vel_conv=1000) # m/s->mm/s
env.units = "mm"
env.center_cell_regrid(periodic_dim=(True, True, True)) # before STL
env.read_stl_mesh_data("obstacle.stl")
env.tile_flow(x=13)        # 13 copies of flow + mesh in x
env.plot_envir()                          # verify before moving

swarm = planktos.Swarm(envir=env, seed=7) # 100 agents
swarm.shared_props["cov"] *= 0.25  # cov = 0.25 I mm^2/s

for _ in range(120):
    swarm.move(0.5, ib_collisions="sticky")

swarm.plot_all()
```

1. Build the world.
2. Load or generate flow.
3. Load geometry and inspect it.
4. Create the Swarm.
5. Advance in a loop.
6. Analyze or export.

The `periodic_dim` flags copy the example's choice for its COMSOL data; match them to your own solver's boundary conditions.

The default `cov` is the identity, so `*= 0.25` gives a standard deviation of $0.5\,\mathrm{mm}/\sqrt{\mathrm{s}}$ per axis.

The COMSOL and STL readers are 3D-only, and immersed meshes must not intersect periodic domain boundaries. The 100 agents start at random in the whole tiled domain (a few may land inside a cylinder); the repository example releases 1000 in a sheet.

API pattern follows `examples/ex_sticky_seafan_3d.py` (Planktos, commit `a1c2128`); a 2D route is `set_brinkman_flow` + `read_IB2d_mesh_data`, see chapter 35.

VELOCITY FIELDS: WHERE THEY COME FROM

Imported data

- IB2d VTK output;
- 3D IBAMR VTK data;
- COMSOL VTU data;
- NetCDF fields;
- arrays supplied directly on a rectilinear grid.

Analytic or transformed data

- Brinkman, two-layer channel, and canopy profiles;
- tiling and simple domain extension;
- cell-center regridding when a solver stores values away from domain edges.

Separation of responsibilities

The CFD or experimental pipeline constructs $\boldsymbol{u}(\boldsymbol{x}, t)$; Planktos samples it along agent paths.

INTERPOLATION: EULERIAN GRID TO AGENT POSITION

For an agent at $\boldsymbol{X}_i(t)$, Planktos evaluates

$$\boldsymbol{u}_i(t) = \mathcal{I}_x[\mathcal{I}_t[\boldsymbol{u}]]\,(\boldsymbol{X}_i(t), t).$$

In time

A SciPy cubic spline is built along the time axis. Outside the recorded interval, the implementation uses constant endpoint values rather than unconstrained spline extrapolation.

In space

The current Swarm drift uses linear `scipy.interpolate.interpn` interpolation on the supplied rectilinear grid; other supported methods can be requested through `interpolate_flow`.

Implementation: current `fluid.py` (`fCubicSpline`), `Environment.interpolate_temporal_flow` (the constant ends) and `Environment.interpolate_flow`.

FLOW-DATA CHECKS THAT PREVENT SILENT MISTAKES

1. **Axis order:** components are stored as $[u_x, u_y, (u_z)]$ on coordinate arrays that increase with the matching indices.
2. **Time labels:** make sure every flow snapshot has the correct physical time.
3. **Units:** `Environment.units` is descriptive; it does not automatically convert velocity, gravity, density, or mesh coordinates.
4. **Cell centers versus corners:** use regridding before loading immersed meshes when needed.
5. **Coverage:** inspect the velocity field and geometry together before advancing agents.

Verification question

If a passive tracer follows a uniform flow, does it travel the expected distance $U\,t$ and obey the intended boundaries?

THREE ROUTES TO BEHAVIOR

Itô SDE

Random walks, biased diffusion, or stochastic decisions.

Euler–Maruyama via `Euler_brownian_motion`.

ODE

Smooth deterministic, tracer, or selected inertial dynamics.

Adaptive RK45 via SciPy `solve_ivp`.

Custom algorithm

State machines, alignment, intermittent search, or hybrid rules.

Override `apply_agent_model`.

A model may be hybrid

For example, a user-supplied ODE can define the drift of an SDE, while `after_move` updates state after boundary and collision handling.

SDE ROUTE: KNOW WHAT THE COVARIANCE MEANS

The built-in stochastic step targets the Itô system

$$d\boldsymbol{X}_t = \boldsymbol{\mu}(\boldsymbol{X}_t, t)\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t,$$

with $\boldsymbol{\sigma}$ constant in space (it may still differ from agent to agent). For a step Δt,

$$\Delta \boldsymbol{X} \approx \boldsymbol{\mu}\, \Delta t + \boldsymbol{\sigma}\sqrt{\Delta t}\, \boldsymbol{\xi}, \qquad \boldsymbol{\xi} \sim \mathcal{N}(\mathbf{0}, I).$$

- The covariance of the stochastic increment is $\boldsymbol{\sigma}\boldsymbol{\sigma}^T \Delta t$.
- In the default drift–diffusion model, the local fluid velocity contributes to the drift.
- Confirm whether a biological paper reports diffusivity, increment variance, or a standard deviation; they differ by units and factors such as $2D$.
- Planktos accepts either a `'cov'` property ($\boldsymbol{\sigma}\boldsymbol{\sigma}^T$) or a `'D'` property (converted as $\texttt{cov} = 2D$). But every Swarm gets a default `'cov'` (the identity), and `'cov'` wins: a `'D'` takes effect only after `del s.shared_props['cov']` (1.0.3, tested).

Docstring of `motion.Euler_brownian_motion` (commit `a1c2128`).

SDE ROUTE: WHEN THE NOISE DEPENDS ON POSITION

For a scalar Itô SDE $dX = \mu\, dt + \sigma(X)\, dW$ with $\Delta W \sim \mathcal{N}(0, \Delta t)$:

$$\underbrace{X_{n+1} = X_n + \mu\Delta t + \sigma\Delta W}_{\text{Euler–Maruyama}} + \underbrace{\tfrac{1}{2}\sigma\sigma'\big[(\Delta W)^2 - \Delta t\big]}_{\text{Milstein correction}}.$$

- If $\sigma' = 0$ (additive noise, the built-in case) the correction vanishes and the two schemes coincide.
- The Stratonovich form of the same dynamics has drift $\mu - \frac{1}{2}\sigma\sigma'$; the conventions agree only when $\sigma' = 0$.

Spatially varying diffusivity

A spatially varying $\boldsymbol{\sigma(X)}$ needs a Milstein-type scheme to keep strong order 1; the built-in Euler–Maruyama step is only (strong) order 0.5 in that case and does not support it directly. (With constant, additive noise, Euler–Maruyama is already strong order 1.) With multiplicative noise the Itô and Stratonovich interpretations also differ.

Docstring of `motion.Euler_brownian_motion` (commit `a1c2128`); schemes and convergence orders: Kloeden & Platen (1992), Ch. 10 (Euler §10.2, Milstein §10.3).

ODE ROUTE: TRACER AND INERTIAL DYNAMICS

Tracer model

$$\dot{\boldsymbol{X}} = \boldsymbol{u}(\boldsymbol{X}, t).$$

This is appropriate only when particle response and active motion are negligible for the target observable.

Inertial or behavioral ODE

$$\dot{\boldsymbol{y}} = \boldsymbol{f}(t, \boldsymbol{y}; \text{flow, state}).$$

Planktos provides generator functions for selected particle models and accepts user-defined systems.

Solver choice

The current `RK45` helper wraps SciPy's adaptive `solve_ivp`; tolerances should be tested against trajectory and ensemble observables.

Current implementation: `planktos/motion.py`. Built-in inertial model: linearized Maxey & Riley (1983) equation in the form of Haller & Sapsis (2008); `RK45` is SciPy's Dormand–Prince (1980) pair.

ODE ROUTE: THE BUILT-IN INERTIAL-PARTICLE MODEL

`motion.inertial_particles(swarm)` returns the right-hand side of the linearized Maxey–Riley equation as Haller & Sapsis (2008) write it:

$$\dot{\boldsymbol{v}} = \tfrac{3R}{2}\frac{D\boldsymbol{u}}{Dt} - \frac{R}{\mathrm{St}}(\boldsymbol{v}-\boldsymbol{u}) + \Big(1-\tfrac{3R}{2}\Big)\boldsymbol{g}, \qquad R = \frac{2\rho_f}{\rho_f+2\rho_p}, \quad \mathrm{St} = \tfrac{2}{9}\Big(\frac{a}{L}\Big)^2\mathrm{Re}.$$

- R carries the density: $R<2/3$ heavier than the fluid, $R=2/3$ neutral, $R>2/3$ bubbles.
- $\mathrm{St}=\tau_p U/L$ measures the particle's response time in units of L/U. In physical units the drag relaxes $\boldsymbol{v}-\boldsymbol{u}$ at the rate $R/\tau_p=(R/\mathrm{St})\,U/L$, $\tau_p=2a^2/(9\nu)$: heavier particles (smaller R) respond more slowly.

Units in 1.0.3

The code uses R/St itself as the drag rate, with dimensional velocities, times and $\boldsymbol{g}$: the two rates agree only when `U/char_L` $=1\,\mathrm{s}^{-1}$. Measured on a neutrally buoyant sphere at rest in a uniform stream ($a=0.1$ mm, $\nu=10^{-6}$ m^2/s, $U=0.1$ m/s): its initial acceleration came out 10, 1 and 0.1 times the physical $RU/\tau_p=30$ m/s^2 for `char_L` $=1,\,0.1$ and 0.01 m.

`motion.inertial_particles` and its docstring (commit `a1c2128`); Maxey & Riley (1983); Haller & Sapsis (2008); measured by calling the returned function.

CUSTOM BEHAVIOR: THE CURRENT EXTENSION POINT

```python
import numpy as np
import planktos

class SeekingSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        flow = self.get_fluid_drift()
        target = self.get_prop("target")
        direction = target - self.positions
        norm = np.linalg.norm(direction, axis=1,
                              keepdims=True)
        direction /= np.maximum(norm, 1e-12)

        drift = flow + self.get_prop("speed")[:,None] \
                     * direction
        return planktos.motion.Euler_brownian_motion(
            self, dt, mu=drift)
```

- Return proposed new positions.
- Let `move` apply outer and immersed-boundary rules afterward.
- Use `after_move` when state depends on the final, collision-corrected position.
- `speed` must be a `props` column (one value per agent): for a shared value `get_prop` returns a float, and `[:,None]` raises `TypeError` (1.0.3, tested).

Current API uses `apply_agent_model`; the 2022 paper used the historical name `get_positions`.

OUTER BOUNDARIES ARE AGENT RULES

Each coordinate direction can use:

`zero`	an exiting agent is masked and no longer updated or plotted;
`noflux`	motion through the wall is projected tangentially, producing a sliding collision;
`periodic`	the agent exits one face and re-enters through the opposite face.

Distinguish two boundary systems

The CFD boundary conditions used to generate u and the agent boundary rules in Planktos are separate. They should be physically compatible, but one does not automatically set the other.

IMMERSED GEOMETRY: BARRIERS INSIDE THE DOMAIN

- 2D meshes are line segments; 3D meshes are triangles.
- Current readers include IB2d-style vertices and 3D STL geometry; 3D vertex data can also be converted through a convex hull workflow (`.vtk` input; in v1.0.3 the `.vertex` branch of that reader drops z and fails, open issue #75).
- Static meshes are supported in 2D and 3D; current code also supports time-varying 2D meshes.
- Mesh segments should not cross except at intended shared vertices and should not intersect periodic domain boundaries.

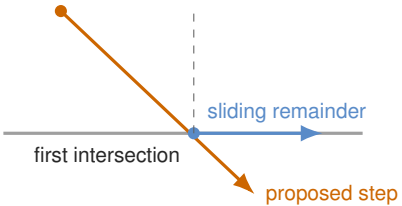
Always inspect

Call `Environment.plot_envir()` after import. Collision logic cannot repair an invalid or topologically ambiguous mesh.

Version note

Moving 2D boundaries are a post-2022 capability in the current repository; 3D meshes remain static.

WHAT HAPPENS AT A COLLISION?



Resolution matters

Large time steps can make collision searches expensive and obscure contact-time physics. Test step-size sensitivity for capture outcomes.

After every step, `Swarm.ib_collision_idx[i]` holds the index of the first mesh element agent i struck in that step, or -1 if it struck none. Exception (1.0.3, tested): a sliding hit followed in the same step by a periodic wrap or a `noflux` wall reads -1; a sticky hit stops the agent and is kept.

sliding	remove motion into the solid and project the remaining displacement along the boundary; recurse if another segment is encountered;
sticky	stop at first contact for that step (backed off the surface by a tiny ε, recorded velocity $= 0$); model persistent attachment with agent state;
`None`	ignore immersed meshes;
`'default'`	use the Swarm's `ib_condition` (default `'sliding'`).

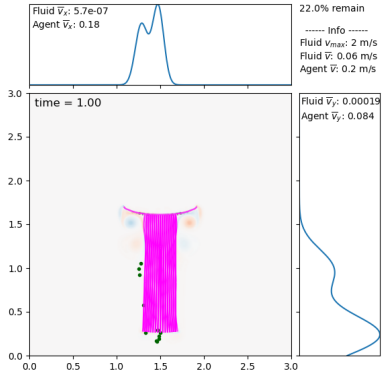
Schematic: instructor's course notes; collision rules from `planktos/_ibc.py` and `Swarm.move`, commit `a1c2128`.

MOVING MESHES AND STICKY ENCOUNTERS

- The current code linearly interpolates supported 2D mesh vertices between stored times.
- Collision handling accounts for mesh motion over the step.
- A user state variable can turn a one-step sticky contact into persistent settlement or capture.

Current limitation

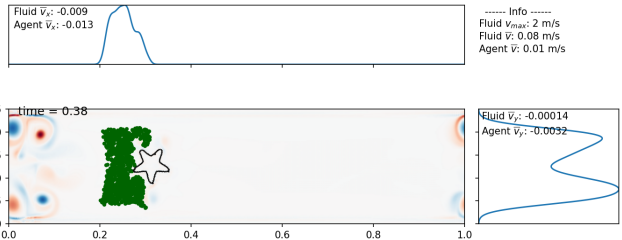
FTLE calculations can respect static immersed boundaries, but moving meshes are not supported in that diagnostic.



Planktos documentation image `docs/_static/ib2d_mvbnd_sticky.png` (commit `a1c2128`, GPLv3).

VISUALIZATION AND POPULATION SUMMARIES

- `plot` gives snapshots; `plot_all` makes animations.
- Position histories export to CSV, or to VTK point data (PolyData) for ParaView/VisIt; `save_data` writes `shared_props` to .npz and `props` to JSON (plus position history and current velocity/acceleration as CSV; `props_history` is not saved).
- Vorticity or velocity may appear behind the agents (2D plots only).
- Marginal Gaussian KDEs summarize position distributions (SciPy `gaussian_kde`; Scott's bandwidth factor by default).



Planktos's own plot: 1000 agents released just upstream of chapter 4f's starfish, $t = 0.38$ s, over the vorticity; the KDE of y has two peaks, one for each side of the starfish.

KDE is an estimate

Bandwidth and boundary bias can change the apparent concentration; report the estimator settings.

Image: this series' run, `examples/Planktos/starfish_kde.py` (Planktos 1.0.3; chapter 4f's IB2d starfish channel). Times on this chapter's plots are Planktos's: its clock starts at the run's dump 1 ($t = 0.02$ s), so $t = 0.38$ s is the end of the run's 0.4 s. Strickland et al. (2022), Fig. 4 shows the same kind of plot for agents in an IB2d flow past a cylinder (not reproduced: the article is not openly licensed).

FTLE: SENSITIVITY OF A DETERMINISTIC FLOW MAP

For a deterministic map $\mathbf{\Phi}_{t_0}^{T}$ with end time T and integration duration $\tau = T - t_0$, define

$$\boldsymbol{C} = (D\mathbf{\Phi})^T D\mathbf{\Phi}, \qquad \sigma_{t_0}^{\tau}(\boldsymbol{X}_0) = \frac{1}{\tau} \ln \sqrt{\lambda_{\max}(\boldsymbol{C})}.$$

- FTLE measures finite-time separation of initially nearby trajectories. Note $\ln \sqrt{\lambda_{\max}} = \frac{1}{2} \ln \lambda_{\max}$: $\sqrt{\lambda_{\max}}$ is the largest stretch factor of an infinitesimal ball.
- Ridges may indicate candidate repelling transport structures, but ridge identification and LCS interpretation require care. Forward-time ridges $\rightarrow$ candidate repelling structures; backward-time ridges $\rightarrow$ attracting; `backward=True` is tracer-only in the current API.
- The diagnostic assumes independent deterministic agent dynamics on the initial-condition grid; ordinary stochastic realizations do not define a single smooth flow map.

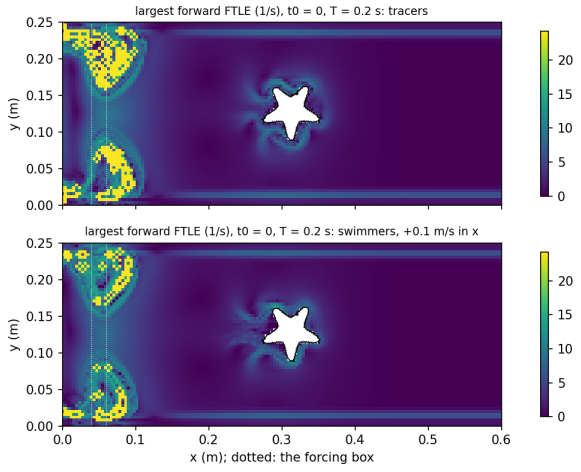
Definition and scope: Strickland et al. (2022), Sections 2.1 and 4.2; current API documentation. FTLE/LCS background: Shadden et al. (2005); Haller (2015).

FTLE IN PLANKTOS: WHAT THE IMPLEMENTATION COMPUTES

- Agents are seeded on a grid (`grid_dim`); $D\Phi$ is approximated by central differences of the end positions, one-sided where a neighbor is masked.
- Planktos normalizes by the time actually integrated (changelog 1.0.3): at each grid point, τ is cut short to the last time before any finite-difference neighbor left the domain.
- In Planktos, forward-time FTLE can use tracers, selected ODEs, or deterministic custom behavior; backward-time support is more restricted.
- `T` is the end time (despite the docstring's "integration time"); the call requires `T > t0`. With `backward=True` the map runs from `t0` back to `2*t0-T`.
- `FTLE_largest` uses $\lambda_{\max}$; `FTLE_smallest` uses $\lambda_{\min}$ of the same forward map (a contraction rate, not backward-time FTLE).
- Static immersed meshes are respected (sliding projection); moving meshes are refused.

Implementation: `Environment.calculate_FTLE`, `planktos/_environment.py`, commit `a1c2128`; `changelog.txt`, v1.0.3.

FTLE CAN REVEAL BEHAVIOR-DEPENDENT TRANSPORT BARRIERS



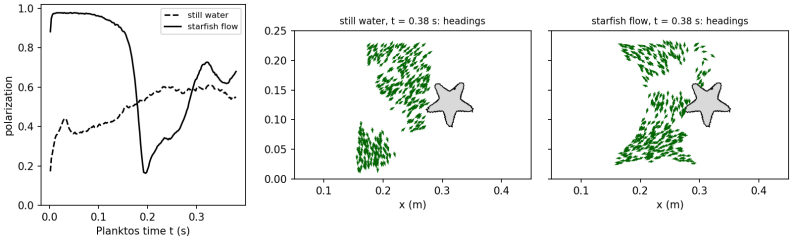
Here: the largest forward-time FTLE over the starfish flow's pulse ($t_0 = 0$, $T = 0.2$ s of Planktos time, the run's 0.02–0.22 s; 256×64 starting points), for tracers (top) and for swimmers that add 0.1 m/s in x to the fluid's velocity (bottom). The ridges around the starfish and in the jet's vortices move: the map is the behaviour's as much as the flow's.

In the paper (Fig. 5): parasitoid-wasp agents fly toward a detected target cylinder in left-to-right flow.

- Faster active flight makes success primarily detection-limited.
- Slower active flight introduces a transport barrier inside the detection radius.

Image: this series' run, `examples/Planktos/starfish_ftle.py` (Planktos 1.0.3; 140 s per map). Interpretation of the wasps: Strickland et al. (2022), Fig. 5 (not reproduced: the article is not openly licensed).

COLLECTIVE RULES CAN INTERACT STRONGLY WITH FLOW



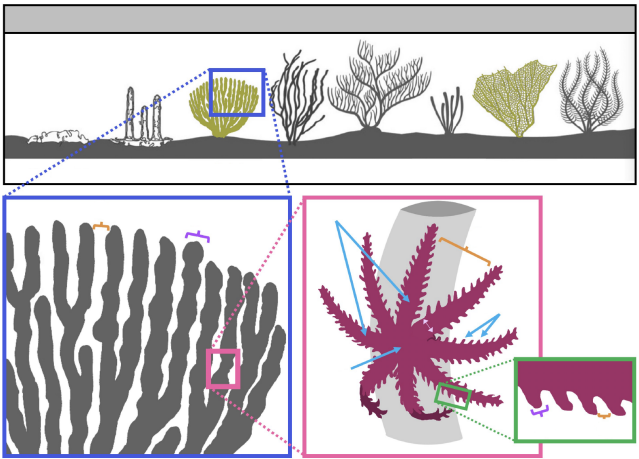
- Here: a Vicsek-type alignment model (Vicsek et al. 1995: each agent turns to its neighbours' mean direction of motion, plus noise; here that motion includes the fluid's), 400 swimmers at 0.1 m/s, neighbours within 2 cm, released upstream of chapter 4f's starfish with random headings. In still water the polarization grows from 0.17 to 0.55 in 0.38 s. In the pulsed flow it is 0.98 at once (the stream dominates the neighbours' mean velocity), falls below 0.2 as the stream near the agents dies away, and recovers to 0.68.
- In the paper (Fig. 9): the same kind of rule, extended to 3D, in a periodic wind tunnel with a cylinder produces coherent, directional structure. Passive and Brownian agents in the same flow provide distinct baselines.

Lesson

A behavior calibrated in still fluid need not produce the same spatial statistics in a heterogeneous flow.

Image: this series' run, `examples/Planktos/starfish_vicsek.py` (Planktos 1.0.3; chapter 35's capstone rule). Strickland et al. (2022), Fig. 9 (not reproduced: the article is not openly licensed).

APPLICATION: FLOW THROUGH A GORGONIAN HIERARCHY



Gorgonians present nested flow filters:

- colony and branches;
- polyps and tentacles;
- pinnules on the tentacles.

At each level, gap-to-diameter ratio and Reynolds number set leakiness. Branches and tentacles can span a leaky-to-solid transition; pinnules mostly impeded flow in the tested range.

Image: Hamlet et al. (2023), Fig. 1 (adapted from Kupfner Johnson & Hallock, 2020), CC BY 4.0.

FROM CFD TO A PLANKTOS CAPTURE EXPERIMENT (1/2): THE PAPER'S

The sea-fan experiment of the 2022 paper (its Fig. 8, not reproduced here):

1. Solve steady flow around one cylinder with periodic lateral boundaries.
2. Tile the field to represent an array of idealized branches.
3. Release agents upstream.
4. Add unbiased Brownian motion to local fluid advection.
5. Treat contact with a cylinder as capture.

In Planktos 1.0.3

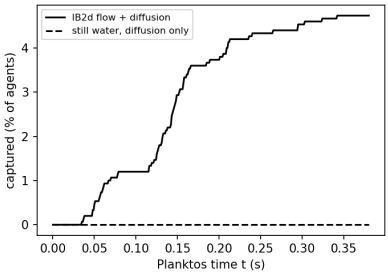
`examples/ex_sticky_seafan_3d.py` of the repository: a COMSOL cylinder tiled into an array (`tile_flow`), sticky collisions (its data from a link in the script). Chapter 35's `CaptureSwarm` keeps a `captured` state.

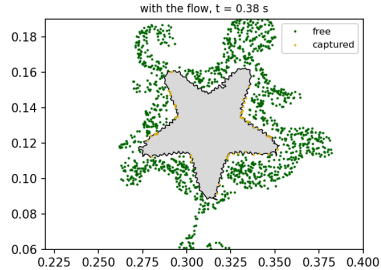
The paper's result (Table 2; 1000 agents, 100 repetitions): 54.3 % captured at $G/D = 0.5$, 31.8 % at $G/D = 2$ (both at 8 mm/s), 27.4 % at $G/D = 2$ and 1 mm/s; sea fans in faster flows, it suggests, should have narrower gaps.

Steps 4 and 5 on this series' starfish, in its own unsteady 2D flow (no tiling; the agents start all around it): next slide.

Strickland et al. (2022), Fig. 8 and its Section 4.5 (Table 2); Planktos `examples/ex_sticky_seafan_3d.py` (commit `a1c2128`).

FROM CFD TO A PLANKTOS CAPTURE EXPERIMENT (2/2): THE STARFISH





- 1500 agents start within 1.5 cm of chapter 4f's starfish, carried by the IB2d flow with a little diffusion ($D = 10^{-6}$ m^2/s); touching the starfish is capture (sticky, and a `captured` state).
- With the flow 4.7 % are captured by $t = 0.38$ s, on most of its faces, the arms' included (gold); in still water, with the same diffusion and the same seed, none. No agent starts closer than about 2 mm (one grid cell), and diffusion alone moves one about 0.9 mm in 0.38 s ($\sqrt{2Dt}$): the flow, not the diffusion, brings them to the surface on this time scale.

This series' run: `examples/Planktos/starfish_capture.py` (Planktos 1.0.3, 90 s).

THE 2023 GORGONIAN CAPTURE STUDY

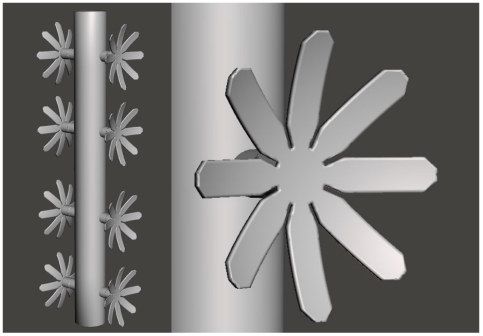


Fig. 6A: a branch with four idealized polyps on each side, from the paper's study of how polyps slow the flow ($G_b/D_b = 2$); the capture runs (Fig. 7) used plain cylinders.

- COMSOL fields were tiled into a $3.6 \times 10 \times 4.0$ cm domain containing an idealized periodic branch array.
- Each case used 1000 agents and 100 repeated simulations.
- Agents followed local flow plus unbiased Brownian motion; cylinder contact counted as capture.
- For $\mathrm{Re}_b = 8$, capture peaked near $G_b/D_b = 1$; for $\mathrm{Re}_b = 1$, it peaked near $G_b/D_b = 1.75$.
- Capture fell when gaps were too narrow or too wide: morphology controlled both delivery and residence near the boundary.

Image: Hamlet et al. (2023), Fig. 6A, CC BY 4.0; results from Fig. 7.

INTERPRET THE APPLICATION WITHIN ITS ASSUMPTIONS

What the model supports

- capture depends jointly on morphology and flow regime;
- an intermediate gap can balance transport through the array with time near the capture surface;
- agent-based simulation connects resolved flow to encounter statistics.

What remains outside it

- deformable or swaying colonies and fully coupled FSI;
- finite-size prey and hydrodynamic feedback;
- adhesion force, contact duration, and post-contact escape;
- unsteady environmental forcing and whole-colony scale separation.

Modeling principle

Treating contact as capture is an operational rule whose biological adequacy must be tested, not a universal law; the 2023 paper itself asks, at higher Re_b, for “the time and force required to capture”.

Caveats: Hamlet et al. (2023), Discussion and Results (capture physics; the quotation); finite size and feedback: this chapter’s one-way contract.

TAKEAWAYS AND SOURCE TRAIL

- Planktos couples **prescribed flow**, **agent behavior**, and **immersed geometry** without agent-to-fluid feedback.
- `Environment` owns Eulerian fields and geometry; `Swarm` owns vectorized Lagrangian state and behavior.
- Flow interpolation, collision rules, and stochastic conventions are part of the mathematical model and should be verified explicitly.
- KDE describes an ensemble distribution; FTLE describes sensitivity of a deterministic finite-time map. They answer different questions.
- The gorgonian study shows the intended workflow: CFD $\rightarrow$ trajectory model $\rightarrow$ capture statistics $\rightarrow$ qualified biological inference.
- This series' flows go through Planktos too: IB2d output directly, IBAMR output after VisIt's resampling and `vtk_for_planktos.py` (`examples/Planktos`; chapter 35).

Primary sources: Strickland et al., *Bull. Math. Biol.* 84:72 (2022); Hamlet et al., *J. Exp. Biol.* 226, jeb244520 (2023), CC BY 4.0.

Software source audited: Planktos, version 1.0.3, commit `a1c2128` (2026-08-19), GPLv3.

REFERENCES

- W. C. Strickland, N. A. Battista, C. L. Hamlet, and L. A. Miller, “Planktos: an agent-based modeling framework for small organism movement and dispersal in a fluid environment with immersed structures,” *Bull. Math. Biol.* 84 (2022), 72, doi:10.1007/s11538-022-01027-1. Source of the deck’s framing; its Figs. 4, 5, 8 and 9 are described, not reproduced (the article is not openly licensed): the pictures beside them are this series’ runs (`examples/Planktos`).
- C. L. Hamlet, W. C. Strickland, N. Battista, and L. A. Miller, “Multiscale flow between the branches and polyps of gorgonians,” *J. Exp. Biol.* 226(5) (2023), jeb244520, doi:10.1242/jeb.244520. CC BY 4.0; Figs. 1 and 6A reproduced, Fig. 7 results summarized.
- W. C. Strickland *et al.*, *Planktos* agent-based modeling framework, v1.0.3, commit `a1c2128` (2026-08-19), GPLv3, https://github.com/mountaindust/Planktos; documentation https://planktos.readthedocs.io. Source of the API statements, the example listing (`examples/ex_sticky_seafan_3d.py`), and the `docs/_static` image (`ib2d_mvbnd_sticky.png`).
- S. Kupfner Johnson and P. Hallock, “A review of symbiotic gorgonian research in the western Atlantic and Caribbean with recommendations for future work,” *Coral Reefs* 39 (2020), 239–258, doi:10.1007/s00338-020-01891-0. Original source adapted in Hamlet et al. (2023), Fig. 1.
- M. R. Maxey and J. J. Riley, “Equation of motion for a small rigid sphere in a nonuniform flow,” *Phys. Fluids* 26 (1983), 883–889, doi:10.1063/1.864230.

REFERENCES (CONTINUED)

- G. Haller and T. Sapsis, “Where do inertial particles go in fluid flows?” *Physica D* 237 (2008), 573–583, doi:10.1016/j.physd.2007.09.027.
- T. Vicsek, A. Czirók, E. Ben-Jacob, I. Cohen, and O. Shochet, “Novel type of phase transition in a system of self-driven particles,” *Phys. Rev. Lett.* 75 (1995), 1226–1229, doi:10.1103/PhysRevLett.75.1226.
- S. C. Shadden, F. Lekien, and J. E. Marsden, “Definition and properties of Lagrangian coherent structures from finite-time Lyapunov exponents in two-dimensional aperiodic flows,” *Physica D* 212 (2005), 271–304, doi:10.1016/j.physd.2005.10.007.
- G. Haller, “Lagrangian coherent structures,” *Annu. Rev. Fluid Mech.* 47 (2015), 137–162, doi:10.1146/annurev-fluid-010313-141322. Caveats on reading FTLE ridges as LCS.
- P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*. Springer, 1992, doi:10.1007/978-3-662-12616-5. Euler–Maruyama and Milstein schemes and their strong orders.
- J. R. Dormand and P. J. Prince, “A family of embedded Runge–Kutta formulae,” *J. Comput. Appl. Math.* 6 (1980), 19–26, doi:10.1016/0771-050X(80)90013-3. The RK45 pair behind SciPy’s `solve_ivp`.

REFERENCES (CONTINUED)

- D. W. Scott, *Multivariate Density Estimation*. Wiley, 1992, doi:10.1002/9780470316849. Scott's KDE bandwidth factor.
- N. A. Battista, W. C. Strickland, and L. A. Miller, "IB2d: a Python and MATLAB implementation of the immersed boundary method," *Bioinspir. Biomim.* 12 (2017), 036003, doi:10.1088/1748-3190/aa5e08. Source format of the IB2d flow and mesh data.
- M. K. Ozalp *et al.*, "Experiments and agent based models of zooplankton movement within complex flow environments," *Biomimetics* 5 (2020), 2, doi:10.3390/biomimetics5010002. A study made with Planktos (its release 0.1).
- Planktos release notes v0.1–v1.0.3, `changelog.txt` and `README.md`, https://github.com/mountaindust/Planktos/releases (read 17 October 2026).
- "Getting started with Planktos" (Miller lab, 2021): install, tests, examples and citation of 2021.

The flow-and-agents cartoon, the architecture diagram, the series diagram and the collision sketch were drawn for these notes (TikZ); the plots are this series' Planktos runs (`examples/Planktos`), except the moving-mesh picture, Planktos's own documentation image. Planktos issue cited on the immersed-geometry slide: https://github.com/mountaindust/Planktos/issues/75.